Heaven's Light is Our Guide

Rajshahi University of Engineering and Technology

Department of Computer Science & Engineering



LAB REPORT

Course Title : Software Engineering Sessional

Course Code : CSE 3206

Name of the Experiment : Lab 3: Design Pattern Analysis, Implementation and Code Review

Date of Submission : 12.10.2026

Submitted By:

Group : Group 2

Section : C **Series :** 2022

Roll	Name
2203124	Ishtiak Ahmed Anan
2203125	Sayed Shafaque bin Nur
2203126	Labib Shahriar Mahi

Submitted To:

Farjana Parvin

Assistant Professor

Department of Computer Science & Engineering

Rajshahi University of Engineering and Technology (RUET)

Remarks:

PART I: BUILDER DESIGN PATTERN

1. Pattern Name

Builder Pattern

2. Category

Creational Design Pattern

3. Intent

The Builder Pattern separates the construction of a complex object from its internal representation, allowing the exact same construction process to produce diverse types and configurations of an object. It enables incremental, step-by-step assembly rather than forcing clients to pass massive argument lists to overloaded constructors.

4. Problem Statement

In modern software engineering, objects frequently contain many attributes—some mandatory and many optional. In our real-world scenario of a **Custom Desktop Computer Assembly System**, an assembled PC contains: Motherboard, Processor, RAM, Storage (NVMe/SATA), Graphics Card, Power Supply, Liquid/Air Cooling, and Wi-Fi modules.

A conventional constructor-based implementation suffers from:

- **Telescoping Constructor Anti-Pattern:** Overloading constructors with 5, 6, 7, or 8+ parameters becomes completely unmaintainable.
- **Parameter Ordering Confusion:** Consecutive parameters sharing identical types (e.g., strings or integers) lead to silent logical bugs when values are mistakenly swapped.
- **Inconsistent Object State:** Using a parameterless constructor followed by public setters leaves the object in a partially initialized, unstable state while being configured.

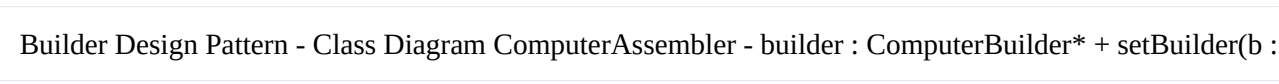
5. Motivation: Why Normal Implementation is Poor

In a direct constructor design, constructing an office PC without a dedicated GPU or liquid cooling forces callers to pass empty strings or null pointers:

```
Computer officePC("MSI B760", "Intel i5", "16GB", "512GB", "", "550W", "Stock Fan", false);
```

Adding new hardware options directly modifies existing constructors, breaking existing client code and violating the **Open/Closed Principle (OCP)**. The Builder Pattern decouples assembly logic into dedicated builder objects, completely resolving this problem.

6. Pattern Structure (UML Class Diagram)



ComputerBuilder*) : void + constructFullPC() : void + constructMinimalPC() : void <<interface>>

ComputerBuilder + reset()* : void + buildMotherboard()* : void + buildCPU()* : void + buildRAM()* : void

+ buildStorage()* : void + buildGPU()* : void + buildPowerSupply()* : void + getResult()* :

unique_ptr<Computer> Computer (Product) - cpu : string - ram : string - storage : string - gpu : string - psu,

cooling : string + setCPU(), setRAM()... + setGPU(), setStorage()... + displaySpecifications() : void

GamingComputerBuilder - computer : unique_ptr<Computer> + buildCPU() [i9-14900K] + buildGPU()

[RTX 4090 24GB] + buildCooling() [360mm AIO] + getResult() : unique_ptr<Computer>

OfficeComputerBuilder - computer : unique_ptr<Computer> + buildCPU() [i5-13400] + buildGPU() [Intel

UHD 730] + buildCooling() [Stock Air] + getResult() : unique_ptr<Computer> aggregates <<implements>>

«creates»

7. Class Responsibilities

- **`Computer` (Product):** The complex domain object storing hardware specifications and providing `displaySpecifications()`.
- **`ComputerBuilder` (Builder Interface):** Declares pure virtual construction steps (`buildMotherboard()`, `buildCPU()`, `buildGPU()`, etc.) and `getResult()`.
- **`GamingComputerBuilder` (Concrete Builder 1):** Assembles top-tier gaming components (Intel i9-14900K, RTX 4090 24GB, 64GB DDR5, 360mm AIO cooler).
- **`OfficeComputerBuilder` (Concrete Builder 2):** Assembles cost-effective office components (Intel i5-13400, integrated UHD 730, stock air cooling).
- **`ComputerAssembler` (Director):** Encapsulates standard assembly workflows (`constructFullPC()`, `constructMinimalPC()`).
- **`Client` (`main`):** Configures the Director with the appropriate Builder, initiates assembly, and consumes the final product.

8. Code Implementation (C++17)

```
/**
 * CSE 3206 Software Engineering Sessional
 * Lab 3: Design Pattern Analysis, Implementation and Code Review
 * Group 2: Builder Pattern Implementation
 *
 * Real-world Scenario: Custom Desktop Computer Assembly System
 */

#include
#include
#include
#include

// ============================================
// 1. PRODUCT
// The complex object under construction.
// ============================================
class Computer {
private:
    std::string cpu;
    std::string motherboard;
    std::string ram;
    std::string storage;
    std::string gpu;
    std::string powerSupply;
    std::string cooling;
    bool hasWifiCard = false;

public:
```

```
    // Setters used by the Builder
    void setCPU(const std::string& val) { cpu = val; }
    void setMotherboard(const std::string& val) { motherboard = val; }
    void setRAM(const std::string& val) { ram = val; }
    void setStorage(const std::string& val) { storage = val; }
    void setGPU(const std::string& val) { gpu = val; }
    void setPowerSupply(const std::string& val) { powerSupply = val; }
    void setCooling(const std::string& val) { cooling = val; }
    void setWifi(bool val) { hasWifiCard = val; }

    void displaySpecifications() const {
        std::cout << "-------------------------------------------------\n";
        std::cout << "             COMPUTER SPECIFICATIONS             \n";
        std::cout << "-------------------------------------------------\n";
        std::cout << "  Motherboard : " << motherboard << "\n";
        std::cout << "  Processor   : " << cpu << "\n";
        std::cout << "  RAM         : " << ram << "\n";
        std::cout << "  Storage     : " << storage << "\n";
        std::cout << "  Graphics    : " << gpu << "\n";
        std::cout << "  Power Supply: " << powerSupply << "\n";
        std::cout << "  Cooling     : " << cooling << "\n";
        std::cout << "  Wi-Fi Card  : " << (hasWifiCard ? "Installed" : "None") << "\n";
        std::cout << "-------------------------------------------------\n\n";
    }
};

// ========================================
// 2. BUILDER INTERFACE
// Declares step-by-step construction methods.
// ========================================
class ComputerBuilder {
```

```
public:

    virtual ~ComputerBuilder() = default;

    virtual void reset() = 0;

    virtual void buildMotherboard() = 0;

    virtual void buildCPU() = 0;

    virtual void buildRAM() = 0;

    virtual void buildStorage() = 0;

    virtual void buildGPU() = 0;

    virtual void buildPowerSupply() = 0;

    virtual void buildCooling() = 0;

    virtual void buildNetwork() = 0;

    virtual std::unique_ptr getResult() = 0;

};



// ========================================

// 3. CONCRETE BUILDER 1: Gaming PC Builder

// High-performance dedicated components.

// ========================================

class GamingComputerBuilder : public ComputerBuilder {

private:

    std::unique_ptr computer;



public:

    GamingComputerBuilder() { reset(); }



    void reset() override {

        computer = std::make_unique();

    }



    void buildMotherboard() override {

        computer->setMotherboard("ASUS ROG Maximus Z790 Hero");
```

```
    }

    void buildCPU() override {
        computer->setCPU("Intel Core i9-14900K (24 cores, 6.0 GHz)");
    }

    void buildRAM() override {
        computer->setRAM("64 GB (2 x 32GB) DDR5-6000 MHz RGB");
    }

    void buildStorage() override {
        computer->setStorage("2 TB Samsung 990 PRO NVMe PCIe 4.0 SSD");
    }

    void buildGPU() override {
        computer->setGPU("NVIDIA GeForce RTX 4090 24GB GDDR6X");
    }

    void buildPowerSupply() override {
        computer->setPowerSupply("Corsair RM1000x 1000W 80+ Gold Fully Modular");
    }

    void buildCooling() override {
        computer->setCooling("NZXT Kraken Elite 360 RGB Liquid Cooler");
    }

    void buildNetwork() override {
        computer->setWifi(true);
    }

    std::unique_ptr getResult() override {
```

```
        std::unique_ptr result = std::move(computer);
        reset(); // Reset builder for next construction
        return result;
    }
};

// ========================================
// 4. CONCRETE BUILDER 2: Office PC Builder
// Cost-effective, reliable business setup.
// ========================================
class OfficeComputerBuilder : public ComputerBuilder {
private:
    std::unique_ptr computer;

public:
    OfficeComputerBuilder() { reset(); }

    void reset() override {
        computer = std::make_unique();
    }

    void buildMotherboard() override {
        computer->setMotherboard("MSI PRO B760M-A WiFi DDR4");
    }

    void buildCPU() override {
        computer->setCPU("Intel Core i5-13400 (10 cores, 4.6 GHz)");
    }

    void buildRAM() override {
        computer->setRAM("16 GB (2 x 8GB) DDR4-3200 MHz");
```

```
    }

    void buildStorage() override {
        computer->setStorage("512 GB Kingston NV2 PCIe 4.0 NVMe SSD");
    }

    void buildGPU() override {
        computer->setGPU("Integrated Intel UHD Graphics 730");
    }

    void buildPowerSupply() override {
        computer->setPowerSupply("Corsair CV550 550W 80+ Bronze");
    }

    void buildCooling() override {
        computer->setCooling("Intel Stock Air Cooler");
    }

    void buildNetwork() override {
        computer->setWifi(true);
    }

    std::unique_ptr getResult() override {
        std::unique_ptr result = std::move(computer);
        reset();
        return result;
    }
};

// ========================================
// 5. DIRECTOR
```

```
// Defines the sequence of construction steps.
// ==========================================
class ComputerAssembler {
private:
    ComputerBuilder* builder;

public:
    void setBuilder(ComputerBuilder* b) {
        builder = b;
    }

    // Construct a complete standard configuration
    void constructFullPC() {
        if (!builder) return;
        builder->reset();
        builder->buildMotherboard();
        builder->buildCPU();
        builder->buildRAM();
        builder->buildStorage();
        builder->buildGPU();
        builder->buildPowerSupply();
        builder->buildCooling();
        builder->buildNetwork();
    }

    // Construct a minimal budget/headless server configuration
    void constructMinimalPC() {
        if (!builder) return;
        builder->reset();
        builder->buildMotherboard();
        builder->buildCPU();
```

```
        builder->buildRAM();
        builder->buildStorage();
        builder->buildPowerSupply();
    }
};

// ========================================
// 6. CLIENT CODE (main)
// Demonstrates how client interacts with Director & Builders.
// ========================================
int main() {
    std::cout << "============================================================\n";
    std::cout << "       CSE 3206 LAB 3 - BUILDER PATTERN DEMO           \n";
    std::cout << "============================================================\n\n";

    ComputerAssembler assembler;

    // 1. Building a High-End Gaming PC via Director
    std::cout << "[Step 1] Assembling High-End Gaming PC via Director...\n";
    GamingComputerBuilder gamingBuilder;
    assembler.setBuilder(&gamingBuilder);
    assembler.constructFullPC();
    std::unique_ptr gamingPC = gamingBuilder.getResult();
    gamingPC->displaySpecifications();

    // 2. Building a Standard Office PC via Director
    std::cout << "[Step 2] Assembling Standard Office Workstation via Director...\n";
    OfficeComputerBuilder officeBuilder;
    assembler.setBuilder(&officeBuilder);
    assembler.constructFullPC();
    std::unique_ptr officePC = officeBuilder.getResult();
```

```
    officePC->displaySpecifications();


    // 3. Custom assembly without Director (Client directly driving Builder)
    std::cout << "[Step 3] Custom Customization: Assembling Custom Barebones Server...\n";
    officeBuilder.reset();
    officeBuilder.buildMotherboard();
    officeBuilder.buildCPU();
    officeBuilder.buildRAM();
    officeBuilder.buildStorage();
    officeBuilder.buildPowerSupply();
    // Intentionally skipped GPU, RGB cooling, Wi-Fi card
    std::unique_ptr customServer = officeBuilder.getResult();
    customServer->displaySpecifications();


    std::cout << "Builder Pattern demonstration executed successfully.\n";
    return 0;
}
```

9. Execution Flow

1. **Instantiation:** Client creates a Director (`ComputerAssembler`) and a Concrete Builder (`GamingComputerBuilder`).
2. **Configuration:** Client assigns the builder to the director via `assembler.setBuilder(&gamingBuilder)`.
3. **Step-by-Step Construction:** Director executes `constructFullPC()`, which calls builder steps in exact logical sequence.
4. **Product Retrieval:** Client calls `gamingBuilder.getResult()`, safely extracting the completed product and resetting the builder.
5. **Director-Free Custom Build:** For custom variations (e.g. headless server), the Client drives builder steps directly without Director intervention.

10. Advantages

- **Fine-Grained Construction Control:** Construction can be executed in distinct steps or deferred until prerequisites are ready.
- **Single Responsibility Principle (SRP):** Isolates complex assembly logic from the domain representation

of the product.

- **Open/Closed Principle (OCP):** New configurations (e.g., `ServerBuilder`) can be added without altering existing code.
- **Eliminates Telescoping Constructors:** Clean, readable code without messy multi-parameter signatures.

11. Limitations

- **Increased Class Count:** Requires defining builder interfaces, concrete builders, and directors, adding architectural overhead for simple objects.
- **Coupling to Product Structure:** Builders must be updated whenever fundamental product structure changes.

12. Real-Life Applications

- Custom PC & Automobile configurators (Dell, Tesla).
- Multi-format document generation engines (compiling into PDF, HTML, or DOCX).
- SQL & HTTP query builders (constructing SELECT, WHERE, JOIN clauses dynamically).

13. Industry Examples

- **Java Core:** `java.lang.StringBuilder`, `java.nio.ByteBuffer`.
- **Android Framework:** `android.app.AlertDialog.Builder`.
- **Spring Framework:** `UriComponentsBuilder`, `MockMvcRequestBuilders`.
- **Apache Commons:** `RequestBuilder`.

14. Demonstration (Live Execution Output)

```
=======================================================

       CSE 3206 LAB 3 - BUILDER PATTERN DEMO

=======================================================


[Step 1] Assembling High-End Gaming PC via Director...

-------------------------------------------------

             COMPUTER SPECIFICATIONS

-------------------------------------------------

  Motherboard : ASUS ROG Maximus Z790 Hero

  Processor   : Intel Core i9-14900K (24 cores, 6.0 GHz)

  RAM         : 64 GB (2 x 32GB) DDR5-6000 MHz RGB
```

```
  Storage     : 2 TB Samsung 990 PRO NVMe PCIe 4.0 SSD
  Graphics    : NVIDIA GeForce RTX 4090 24GB GDDR6X
  Power Supply: Corsair RM1000x 1000W 80+ Gold Fully Modular
  Cooling     : NZXT Kraken Elite 360 RGB Liquid Cooler
  Wi-Fi Card  : Installed
-------------------------------------------------


[Step 2] Assembling Standard Office Workstation via Director...
-------------------------------------------------
             COMPUTER SPECIFICATIONS
-------------------------------------------------
  Motherboard : MSI PRO B760M-A WiFi DDR4
  Processor   : Intel Core i5-13400 (10 cores, 4.6 GHz)
  RAM         : 16 GB (2 x 8GB) DDR4-3200 MHz
  Storage     : 512 GB Kingston NV2 PCIe 4.0 NVMe SSD
  Graphics    : Integrated Intel UHD Graphics 730
  Power Supply: Corsair CV550 550W 80+ Bronze
  Cooling     : Intel Stock Air Cooler
  Wi-Fi Card  : Installed
-------------------------------------------------


[Step 3] Custom Customization: Assembling Custom Barebones Server...
-------------------------------------------------
             COMPUTER SPECIFICATIONS
-------------------------------------------------
  Motherboard : MSI PRO B760M-A WiFi DDR4
  Processor   : Intel Core i5-13400 (10 cores, 4.6 GHz)
  RAM         : 16 GB (2 x 8GB) DDR4-3200 MHz
  Storage     : 512 GB Kingston NV2 PCIe 4.0 NVMe SSD
  Graphics    :
  Power Supply: Corsair CV550 550W 80+ Bronze
```

```
  Cooling     :

  Wi-Fi Card  : None

------------------------------------------------


Builder Pattern demonstration executed successfully.
```

15. Conclusion

The Builder Pattern is the gold standard for constructing complex aggregate objects with many optional properties. It guarantees encapsulation, prevents invalid intermediary states, and upholds SOLID object-oriented design principles.

PART II: PROTOTYPE DESIGN PATTERN

1. Pattern Name

Prototype Pattern

2. Category

Creational Design Pattern

3. Intent

The Prototype Pattern specifies the kinds of objects to create using a prototypical instance, creating new objects by copying this prototype via a virtual `clone()` method. It shields client code from concrete instantiation classes and eliminates expensive initialization costs.

4. Problem Statement

In real-time software systems such as **Game Engines (RTS / RPG)**, Graphics Editors, and Virtual Reality platforms, creating complex objects from scratch is computationally expensive. Spawning an enemy unit (`Swordsman`, `Archer`, `Mage`) requires reading 3D models and textures from storage, compiling audio waveforms, parsing skill trees from JSON/XML configs, and calculating attribute curves.

Calling `new Swordsman(...)` dozens of times during real-time gameplay creates severe frame drops and stalls. Furthermore, hardcoding instantiation logic couples the spawner directly to concrete unit classes.

5. Motivation: Why Normal Implementation is Poor

- **Expensive Initialization Overhead:** Re-executing file I/O, asset decompression, and attribute parsing for identical units is redundant and slow.
- **Tight Coupling:** Client spawner classes become tightly coupled to concrete unit classes, violating the

Dependency Inversion Principle (DIP).

- **Polymorphic Object Copying:** In C++, if a client holds only an abstract base pointer (`GameUnit*`), it cannot invoke `new Derived()` because the exact derived class is unknown at compile time. The Prototype pattern's virtual `clone()` provides seamless polymorphic copying.

6. Pattern Structure (UML Class Diagram)



7. Class Responsibilities

- **`GameUnit` (Prototype Interface):** Declares pure virtual `clone()` method and unit operations (`render()`, `setPosition()`, `setCustomName()`).
- **`Swordsman` (Concrete Prototype 1):** Stores heavy melee combat data and implements `clone()` via copy constructor.
- **`Archer` (Concrete Prototype 2):** Stores ranged combat data and implements `clone()`.
- **`Mage` (Concrete Prototype 3):** Stores magic spell arrays and implements `clone()`.

- **`UnitRegistry` (Prototype Cache / Spawner):** Maintains a hash map of master pre-initialized prototypes and handles clone requests by key.
- **`Client` (`main`):** Registers master prototypes, spawns clones dynamically on the battlefield, adjusts runtime properties, and deploys units.

8. Code Implementation (C++17)

```
/**
 * CSE 3206 Software Engineering Sessional
 * Lab 3: Design Pattern Analysis, Implementation and Code Review
 * Group 2: Prototype Pattern Implementation
 *
 * Real-world Scenario: Game Character / Unit Spawning Engine (RTS/RPG)
 */


#include
#include
#include
#include
#include


// ========================================
// 1. PROTOTYPE INTERFACE
// Declares the clone() method.
// ========================================
class GameUnit {
public:
    virtual ~GameUnit() = default;


    // Core Prototype clone interface (pure virtual)
    virtual std::unique_ptr clone() const = 0;


    // Presentation / gameplay method
```

```
    virtual void render(int unitId) const = 0;



    // Dynamic runtime customization

    virtual void setPosition(int x, int y) = 0;

    virtual void setCustomName(const std::string& name) = 0;

};



// ========================================

// 2. CONCRETE PROTOTYPE 1: Swordsman / Warrior

// ========================================

class Swordsman : public GameUnit {

private:

    std::string unitType;

    std::string customName;

    int health;

    int attackPower;

    int armor;

    std::string weapon;

    int posX = 0;

    int posY = 0;



public:

    Swordsman(const std::string& name, int hp, int atk, int def, const std::string& wpn)

        : unitType("Swordsman"), customName(name), health(hp), attackPower(atk), armor(def),
weapon(wpn) {

        // Simulating expensive initialization (asset loading, audio preload, skill tree
compilation)

    }



    // Copy constructor used for cloning

    Swordsman(const Swordsman& other)
```

```
        : unitType(other.unitType),
          customName(other.customName),
          health(other.health),
          attackPower(other.attackPower),
          armor(other.armor),
          weapon(other.weapon),
          posX(other.posX),
          posY(other.posY) {}

    // Implement clone() using copy constructor
    std::unique_ptr clone() const override {
        return std::make_unique(*this);
    }

    void setPosition(int x, int y) override {
        posX = x;
        posY = y;
    }

    void setCustomName(const std::string& name) override {
        customName = name;
    }

    void render(int unitId) const override {
        std::cout << "  [Unit #" << unitId << " - " << unitType << "] \"" << customName <<
"\"\n"
                  << "    Stats : HP=" << health << " | ATK=" << attackPower << " | ARMOR="
<< armor << "\n"
                  << "    Weapon: " << weapon << "\n"
                  << "    Coord : (" << posX << ", " << posY << ")\n\n";
    }
```

```
};


// ========================================
// 3. CONCRETE PROTOTYPE 2: Archer
// ========================================
class Archer : public GameUnit {
private:
    std::string unitType;
    std::string customName;
    int health;
    int attackRange;
    int agility;
    std::string bowType;
    int posX = 0;
    int posY = 0;

public:
    Archer(const std::string& name, int hp, int range, int agi, const std::string& bow)
        : unitType("Archer"), customName(name), health(hp), attackRange(range), agility(agi),
bowType(bow) {}

    Archer(const Archer& other)
        : unitType(other.unitType),
          customName(other.customName),
          health(other.health),
          attackRange(other.attackRange),
          agility(other.agility),
          bowType(other.bowType),
          posX(other.posX),
          posY(other.posY) {}
```

```
    std::unique_ptr clone() const override {
        return std::make_unique(*this);
    }

    void setPosition(int x, int y) override {
        posX = x;
        posY = y;
    }

    void setCustomName(const std::string& name) override {
        customName = name;
    }

    void render(int unitId) const override {
        std::cout << "  [Unit #" << unitId << " - " << unitType << "] \"" << customName << "\"\n"
                  << "    Stats : HP=" << health << " | RANGE=" << attackRange << " | AGILITY=" << agility << "\n"
                  << "    Weapon: " << bowType << "\n"
                  << "    Coord : (" << posX << ", " << posY << ")\n\n";
    }
};

// ========================================
// 4. CONCRETE PROTOTYPE 3: Mage / Sorcerer
// ========================================
class Mage : public GameUnit {
private:
    std::string unitType;
    std::string customName;
    int health;
```

```
    int mana;

    std::string spellElement;

    std::string staffType;

    int posX = 0;

    int posY = 0;



public:

    Mage(const std::string& name, int hp, int mp, const std::string& element, const
std::string& staff)

        : unitType("Mage"), customName(name), health(hp), mana(mp), spellElement(element),
staffType(staff) {}



    Mage(const Mage& other)

        : unitType(other.unitType),

          customName(other.customName),

          health(other.health),

          mana(other.mana),

          spellElement(other.spellElement),

          staffType(other.staffType),

          posX(other.posX),

          posY(other.posY) {}



    std::unique_ptr clone() const override {

        return std::make_unique(*this);

    }



    void setPosition(int x, int y) override {

        posX = x;

        posY = y;

    }
```

```
    void setCustomName(const std::string& name) override {
        customName = name;
    }

    void render(int unitId) const override {
        std::cout << "  [Unit #" << unitId << " - " << unitType << "] \"" << customName << "\"\n"
                  << "    Stats : HP=" << health << " | MANA=" << mana << " | ELEMENT=" << spellElement << "\n"
                  << "    Weapon: " << staffType << "\n"
                  << "    Coord : (" << posX << ", " << posY << ")\n\n";
    }
};

// ========================================
// 5. PROTOTYPE REGISTRY / SPAWN MANAGER
// Caches master prototypes and clones them on demand.
// ========================================
class UnitRegistry {
private:
    std::unordered_map> prototypes;

public:
    void registerPrototype(const std::string& key, std::unique_ptr prototype) {
        prototypes[key] = std::move(prototype);
    }

    // Creates a new unit instance by cloning the registered prototype
    std::unique_ptr spawnUnit(const std::string& key) const {
        auto it = prototypes.find(key);
        if (it != prototypes.end()) {
```

```
            return it->second->clone(); // Returns deep cloned copy
        }
        std::cerr << "Error: Prototype with key '" << key << "' not registered!\n";
        return nullptr;
    }
};

// ========================================
// 6. CLIENT CODE (main)
// Demonstrates fast spawning via cloning.
// ========================================
int main() {
    std::cout << "============================================================\n";
    std::cout << "      CSE 3206 LAB 3 - PROTOTYPE PATTERN DEMO          \n";
    std::cout << "============================================================\n\n";

    // 1. Initializing and Registering Master Prototypes in Registry
    std::cout << "[Step 1] Loading and caching master unit prototypes...\n";
    UnitRegistry registry;

    registry.registerPrototype(
        "elite_swordsman",
        std::make_unique("Royal Guard", 150, 45, 30, "Excalibur Greatsword")
    );

    registry.registerPrototype(
        "sniper_archer",
        std::make_unique("Elven Marksman", 90, 80, 70, "Windrunner Longbow")
    );

    registry.registerPrototype(
```

```
        "pyro_mage",
        std::make_unique("Archmage", 80, 200, "Firestorm", "Phoenix Staff")
    );

    std::cout << "Master prototypes successfully registered in memory.\n\n";

    // 2. Spawning Army Squad via Fast Cloning
    std::cout << "[Step 2] Spawning units on battlefield via Prototype cloning:\n";
    std::cout << "-------------------------------------------------------\n";

    std::vector> battlefieldArmy;

    // Clone 2 Swordsmen
    auto s1 = registry.spawnUnit("elite_swordsman");
    s1->setPosition(10, 20);
    s1->setCustomName("Vanguard Arthur");
    battlefieldArmy.push_back(std::move(s1));

    auto s2 = registry.spawnUnit("elite_swordsman");
    s2->setPosition(12, 22);
    s2->setCustomName("Vanguard Lancelot");
    battlefieldArmy.push_back(std::move(s2));

    // Clone 2 Archers
    auto a1 = registry.spawnUnit("sniper_archer");
    a1->setPosition(5, 50);
    a1->setCustomName("Scout Robin");
    battlefieldArmy.push_back(std::move(a1));

    auto a2 = registry.spawnUnit("sniper_archer");
    a2->setPosition(8, 52);
```

```
    a2->setCustomName("Scout Legolas");
    battlefieldArmy.push_back(std::move(a2));

    // Clone 1 Mage
    auto m1 = registry.spawnUnit("pyro_mage");
    m1->setPosition(2, 15);
    m1->setCustomName("Grand Wizard Gandalf");
    battlefieldArmy.push_back(std::move(m1));

    // 3. Render all deployed units
    int unitId = 1;
    for (const auto& unit : battlefieldArmy) {
        unit->render(unitId++);
    }

    std::cout << "------------------------------------------------------\n";
    std::cout << "Prototype Pattern demonstration executed successfully.\n";
    std::cout << "Notice: All 5 units were instantiated via zero-overhead\n";
    std::cout << "cloning without repeating expensive initialization logic!\n";

    return 0;
}
```

9. Execution Flow

1. **Prototype Pre-caching:** During system boot, master unit templates are created once and stored in the `UnitRegistry`.
2. **Clone Request:** At runtime, the client requests a unit: `registry.spawnUnit("elite_swordsman")`.
3. **Virtual Clone Execution:** The registry finds the master instance and calls `clone()`, invoking the derived copy constructor.
4. **Dynamic Mutation:** The client configures instance-specific fields (e.g., coordinates, unit nicknames) without mutating the master prototype.
5. **Deployment:** The freshly cloned unit is added to the active unit list with zero initialization delay.

10. Advantages

- **Blazing Fast Object Creation:** Bypasses expensive asset loading, constructor calculations, and disk/network I/O.
- **Polymorphic Cloning:** Duplicates objects through base pointers without coupling to concrete classes.
- **Runtime Dynamic Configuration:** Prototypes can be registered, replaced, or removed dynamically without recompiling.
- **Reduces Subclass Explosion:** Pre-configured presets can replace deep subclassing hierarchies.

11. Limitations

- **Deep vs. Shallow Copy Nuances:** Cloning objects containing circular references or dynamic raw pointers requires meticulous deep-copy handling to prevent double-free or memory corruption.

12. Real-Life Applications

- Real-Time Strategy / RPG Game enemy and projectile spawning.
- Graphic vector applications (copy-pasting complex vector shapes in Illustrator / Figma).
- Virtual Machine and container provisioning (spawning containers from master image snapshots).

13. Industry Examples

- **Java:** `java.lang.Cloneable` interface and `Object.clone()` method.
- **JavaScript:** Fundamental prototype-based object inheritance model.
- **Unity Game Engine:** `Instantiate(prefab)` cloning system.
- **Unreal Engine:** Class Default Object (CDO) instantiation.

14. Demonstration (Live Execution Output)

```
=========================================================

      CSE 3206 LAB 3 - PROTOTYPE PATTERN DEMO

=========================================================


[Step 1] Loading and caching master unit prototypes...

Master prototypes successfully registered in memory.


[Step 2] Spawning units on battlefield via Prototype cloning:

---------------------------------------------------------

  [Unit #1 - Swordsman] "Vanguard Arthur"
```

```
    Stats : HP=150 | ATK=45 | ARMOR=30
    Weapon: Excalibur Greatsword
    Coord : (10, 20)

  [Unit #2 - Swordsman] "Vanguard Lancelot"
    Stats : HP=150 | ATK=45 | ARMOR=30
    Weapon: Excalibur Greatsword
    Coord : (12, 22)

  [Unit #3 - Archer] "Scout Robin"
    Stats : HP=90 | RANGE=80 | AGILITY=70
    Weapon: Windrunner Longbow
    Coord : (5, 50)

  [Unit #4 - Archer] "Scout Legolas"
    Stats : HP=90 | RANGE=80 | AGILITY=70
    Weapon: Windrunner Longbow
    Coord : (8, 52)

  [Unit #5 - Mage] "Grand Wizard Gandalf"
    Stats : HP=80 | MANA=200 | ELEMENT=Firestorm
    Weapon: Phoenix Staff
    Coord : (2, 15)

------------------------------------------------------
Prototype Pattern demonstration executed successfully.
Notice: All 5 units were instantiated via zero-overhead
cloning without repeating expensive initialization logic!
```

15. Conclusion

The Prototype Pattern provides an optimal mechanism for high-frequency object generation where instantiation

cost is significant. Combined with a centralized Prototype Registry, it maximizes runtime efficiency while maintaining clean object-oriented decoupling.

SUMMARY COMPARISON OF CREATIONAL DESIGN PATTERNS

Feature / Criteria	Builder Pattern	Prototype Pattern	Factory Method	Abstract Factory
Primary Intent	Step-by-step construction of complex multi-part objects	Cloning pre-configured objects via `clone()`	Subclasses decide which class to instantiate	Creates families of related or dependent objects
Object Complexity	High (many configurable parts & options)	Heavy initialization cost, identical initial state	Single product hierarchy	Multiple product families / suites
Construction Mechanism	Step-by-step calls via Director or Fluent Builder API	Copy constructor / virtual `clone()` method	Inheritance and polymorphic method override	Object composition via abstract factory interface
Key Advantage	Eliminates telescoping constructors	Zero expensive re-initialization overhead	Decouples creator from concrete product class	Enforces compatibility among product families
When to Use	Object has many optional parameters & representations	Instantiating from scratch is costly or slow	Object creation logic varies across subclasses	System requires multi-platform component suites